ZenCro K-12 School Management System

Developer Handoff & Frontend ↔ Backend Integration Guide

Two-Repository Development Workflow | Zainab — Backend | Mariya — Frontend | Day 8 Integration

DEVELOPER HANDOFF

Purpose
A practical working agreement for two developers building the K–12 MVP independently while staying synchronized through the existing API/data contract.

Development model
Days 1–7: parallel module development with contract-matched mocks.
Day 8: full frontend ↔ backend integration, system testing, security verification, fixing and deployment.

THE BRIDGE IS THE API / DATA CONTRACT

The two repositories are an implementation boundary, not a product boundary. The repositories may be independent; the agreed contract must remain synchronized.

Authoritative source set

- ZenCro School Management System — Finalized MVP Feature Specification
- ZenCro K12 Backend / Frontend / API Data Contract Specification
- ZenCro School Management System — Detailed Research & Product Blueprint
- ZenCro School Management System — K-12 8-Day MVP Development Plan

This guide is a companion handoff document. It does not replace or redefine the product specification, API/data contract, research blueprint or 8-day development plan.

Contents.

1. Document Purpose
2. Authoritative Document Hierarchy
3. Team Responsibilities
4. Two-Repository Structure
5. Shared API Contract — The Bridge
6. Mock Data → Real API Strategy
7. Core API Conventions
8. Error Response Standard
9. Role and Security Rules
10. Environment Variable Rules
11. Day-by-Day Working Model
12. Daily Communication Rule
13. What Should Not Happen
14. Day 8 Integration Process
15. Cross-Module Data Principle
16. Final Responsibility Matrix
17. One-Page Rules

SOURCE DISCIPLINE

Where this guide repeats API, security or workflow details, the existing project source documents remain authoritative. This handoff operationalizes them; it does not create a competing specification.

1. Document Purpose

✓ Zainab and Mariya develop the K–12 MVP in parallel.
✓ Zainab owns the backend; Mariya owns the frontend.
✓ They work from two separate GitHub repositories.
✓ They do not need to share GitHub passwords or collaborate inside the same repository.
✓ The API/data contract is the bridge between the two repositories.
✓ Days 1–7 are primarily independent development; the frontend uses mock data matching agreed API response shapes.
✓ The backend develops the real database, business logic, APIs, authentication, RBAC and RLS.
✓ Day 8 is the planned full frontend ↔ backend integration and system testing point.

NON-NEGOTIABLE

The two-repository workflow does NOT change the existing API/data contract or product requirements.

2. Authoritative Document Hierarchy

Document	Purpose
MVP Feature Specification	Defines what the product must do.
Backend/Frontend/API Data Contract	Defines how frontend and backend communicate.
Detailed Research & Product Blueprint	Defines product reasoning, architecture, security and research foundation.
8-Day MVP Development Plan	Defines the day-by-day development sequence.
This Developer Handoff Guide	Defines how Zainab and Mariya work independently and integrate on Day 8.

CONFLICT RULE

If there is a conflict, do not silently modify the contract. Discuss the conflict and update the relevant authoritative document/contract before proceeding.

3. Team Responsibilities

Zainab — Backend

- ✓ Supabase project and PostgreSQL database
- ✓ Database schema and migrations
- ✓ Authentication foundation
- ✓ User and school structure
- ✓ Roles, RBAC and Row Level Security
- ✓ Backend APIs and server-side validation
- ✓ Business rules
- ✓ Student APIs and Student 360 aggregation
- ✓ Attendance APIs and offline-sync foundation
- ✓ Fees APIs and calculations
- ✓ Exams/results/report-card APIs
- ✓ Communication APIs
- ✓ Parent relationships and access
- ✓ PTM APIs
- ✓ Forms APIs/storage
- ✓ Import/migration processing
- ✓ Export services
- ✓ Dashboard aggregation APIs
- ✓ Data/privacy logic and audit/security controls
- ✓ API, RLS and permission testing
- ✓ Day 8 backend integration/security testing
- ✓ Deployment/backend environment configuration

Mariya — Frontend

- ✓ Next.js application, React and TypeScript
- ✓ Tailwind CSS and shadcn/ui
- ✓ Navigation and application shell
- ✓ Login/session UI and school setup UI
- ✓ Student management and Student 360 UI
- ✓ Attendance UI and offline/sync indicators
- ✓ Fees UI
- ✓ Exams/results/report-card UI
- ✓ Communication UI
- ✓ Parent dashboard and PTM UI
- ✓ Forms UI
- ✓ Import/export UI
- ✓ Admin and Teacher dashboards
- ✓ Data/privacy UI
- ✓ Loading, empty and error states
- ✓ Responsive/mobile/tablet behavior
- ✓ Mock API services during Days 1–7
- ✓ Real API integration on Day 8
- ✓ Frontend testing and UI bug fixing

OWNERSHIP BOUNDARY

Backend owns data, authorization, business rules and API behavior. Frontend owns presentation, interaction, state display and user experience. The contract is jointly agreed.

4. Two-Repository Structure

```
GitHub
■
■■■ Zainab's Repository
■   ■■■ zencro-school-management-system
■       ■■■ Backend / Next.js Route Handlers / Supabase integration
■
■■■ Mariya's Repository
    ■■■ zencro-school-management-frontend
        ■■■ Frontend / Next.js / React / UI
```

- ✓ Each developer owns and pushes to her own repository.
- ✓ No GitHub password sharing.
- ✓ No collaborator access between repositories is required for the workflow.
- ✓ The repositories are intentionally developed independently.
- ✓ The repositories do not need to be merged every day.
- ✓ The frontend repository consumes the backend through the agreed API contract.
- ✓ Day 8 is the main integration point.
- ✓ Repository consolidation can be considered later if the project requires it; it is not necessary for the MVP workflow.

KEY IDEA

Separate repositories are a workflow choice. They do not create separate product definitions. The API/data contract remains the shared technical boundary.

5. Shared API Contract — The Bridge

Both developers must agree on the following before consuming or implementing an API:

✓ Endpoint names and HTTP methods
✓ Request fields and response fields
✓ Field names and data types
✓ Required, optional and nullable fields
✓ Validation rules
✓ Role permissions and school/tenant scope
✓ Error format
✓ Pagination conventions
✓ Date conventions
✓ Status values
✓ Loading/empty/error expectations
✓ Mock data structure

CONTRACT MISMATCH EXAMPLE

```
Backend: admission_no
Frontend: admissionNumber

Result: CONTRACT MISMATCH
```

A naming mismatch should be corrected before Day 8 rather than discovered during final integration. The same discipline applies to types, nullability, status values, error shapes and permission semantics.

CONTRACT CHANGE RULE

No silent request/response changes. Identify → communicate → update the agreed contract → update mocks → update backend → update frontend → verify.

6. Mock Data → Real API Strategy

Days 1–7

Mariya

✓ Builds UI using mock objects.
✓ Mock objects match the agreed backend response structure exactly.
✓ Develops success, loading, empty and error states independently.

Zainab

✓ Builds the real database and APIs.
✓ Builds validation and business rules.
✓ Builds authentication, RBAC and RLS.
✓ Tests API responses against the agreed contract.

Contract Change

1. Identify the change.
2. Inform the other developer.
3. Update the agreed contract.
4. Update mock data.
5. Update backend API.
6. Update frontend expectations.
7. Verify both sides again.

Day 8

Mariya

✓ Replace mock API calls with real API calls.

✓ Test loading, success, empty and error states.

✓ Perform responsive and user-flow testing.

Zainab

✓ Verify real API responses.

✓ Test authentication, RBAC and RLS.

✓ Test school/tenant isolation, validation and business rules.

✓ Fix backend/API/data issues.

> **IMPORTANT**
>
> Mocks are not throwaway guesses. During Days 1–7 they are the frontend's executable interpretation of the agreed API contract.

7. Core API Conventions

GET `/api/me`

Purpose: Return authenticated identity, role and school context.

```
{
  "user": {
    "id": "...",
    "name": "...",
    "email": "...",
    "role": "...",
    "school_id": "..."
  },
  "school": {
    "id": "...",
    "name": "..."
  }
}
```

✓ Backend derives the authenticated user from the session.

✓ Backend derives school context securely.

✓ Frontend must not be trusted to provide its own school scope.

✓ school_id must not be trusted simply because the client sends it.

Approved role values

```
admin
principal
teacher
accountant
parent
```

> **ROLE SCOPE**
>
> Use only the approved role values above. Do not invent additional roles in the handoff or implementation.

8. Error Response Standard

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Admission number already exists.",
    "field": "admission_no"
  }
}
```

Suggested error codes

```
AUTH_REQUIRED
FORBIDDEN
NOT_FOUND
VALIDATION_ERROR
CONFLICT
INTERNAL_ERROR
```

✓ Backend returns predictable errors.

✓ Frontend displays appropriate user-facing messages.

✓ Validation errors may identify a field.

✓ Secrets and sensitive internal implementation details must not be exposed.

> **FRONTEND EXPECTATION**
>
> The UI should distinguish authentication, authorization, missing records, validation, conflicts and generic internal failures without parsing arbitrary backend prose.

9. Role and Security Rules

```
Authentication
      ↓
Role / RBAC
      ↓
School / Tenant Scope
      ↓
RLS
      ↓
Server-side Validation
      ↓
Business Rules
```

✓ Authentication is required for protected APIs.

✓ Role permissions must be enforced.

✓ School-level tenant isolation must be enforced.

✓ RLS is part of the security model.

✓ Backend validation is mandatory even if frontend validation exists.

✓ Client-side checks are not security controls.

✓ Do not trust client-provided school IDs.

✓ Do not expose Supabase service-role credentials to the browser.

✓ Do not put secret credentials in NEXT_PUBLIC_* environment variables.

✓ Do not commit secrets to GitHub.

> **COMPLIANCE BOUNDARY**
>
> Technical safeguards such as Auth/RBAC/RLS support security and privacy controls, but they are not by themselves a claim of legal DPDP compliance. Compliance remains a product, operational and legal workstream.

10. Environment Variable Rules

Frontend-safe variables may use:

```
NEXT_PUBLIC_...
```

Server-only secrets must NOT use:

```
NEXT_PUBLIC_...
```

Supabase environment concept

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
```

> **SUPABASE_SERVICE_ROLE_KEY = SERVER ONLY**
>
> Never expose it to browser code, return it in API responses, commit it to GitHub, place it in frontend environment variables, or use it inside client components.

✓ Use .env.example for variable names/placeholders.

✓ Use .env.local for actual local secrets.

✓ Keep production secrets in the hosting platform's secure configuration.

✓ Separate development, staging and production environments so testing does not touch real school data.

11. Day-by-Day Working Model

Day	Zainab — Backend	Mariya — Frontend	Integration Status
Day 1	Foundation, Supabase, schema, auth foundation, roles, RLS, API conventions	App shell, navigation, login/session UI, dashboard shell	Independent
Day 2	Student APIs, guardian relationships, Student 360, validation/RLS	Student UI, profile, 360 interface	Independent + contract sync
Day 3	Attendance APIs, business rules, offline-sync foundation	Attendance UI, offline indicators, history	Independent + contract sync
Day 4	Fees APIs, payments, outstanding calculations	Fees dashboard and payment UI	Independent + contract sync
Day 5	Exams, results, report-card backend	Exams/results/report-card UI	Independent + contract sync
Day 6	Communication, parent, PTM, forms APIs	Communication, parent, PTM, forms UI	Independent + contract sync
Day 7	Import/export, dashboards, privacy, audit/security logic	Import/export, dashboards, privacy UI	Feature freeze / integration-ready
Day 8	Integration, API/RLS/security testing, deployment	Real API integration, E2E/UI testing, fixes	FULL INTEGRATION

SEQUENCE RULE

Do not add an integration checkpoint before Day 8. Days 1–7 build and test modules; Day 8 performs full integration, system testing, security verification, fixing and deployment.

12. Daily Communication Rule

Keep communication lightweight. The goal is contract synchronization, not continuous code merging.

Zainab sends

- ✓ APIs completed
- ✓ Contract finalized/changed
- ✓ Request/response changes
- ✓ Important validation rules
- ✓ Role/permission notes
- ✓ Any blocker

Mariya sends

- ✓ UI completed
- ✓ Mock services completed
- ✓ Fields consumed from contract
- ✓ Any expected contract issue
- ✓ Any blocker

COMMUNICATION ≠ MERGING

The two developers can remain in separate repositories while maintaining a shared technical contract. Communication keeps the interface synchronized; it does not require daily repository merging.

13. What Should Not Happen

- ✕ Share GitHub passwords.
- ✕ Randomly copy entire repositories.
- ✕ Modify the API response structure without telling the other developer.
- ✕ Invent new API fields independently.
- ✕ Use different field names for the same concept.
- ✕ Hard-code production credentials.
- ✕ Put service-role secrets in frontend code.
- ✕ Trust client-provided school IDs.
- ✕ Build excluded MVP modules.
- ✕ Integrate continuously just because both repositories exist.
- ✕ Merge backend and frontend code before the planned integration point unless specifically required.
- ✕ Discover major contract differences for the first time on Day 8.

DAY 8 SHOULD NOT BE A SURPRISE

The final integration day is for connecting already-prepared modules and finding real integration issues—not for discovering that the two developers implemented different contracts.

14. Day 8 Integration Process

Step 1 — Freeze the contract

Confirm endpoints, requests, responses, errors and roles.

Step 2 — Backend readiness

Database migrations; Supabase configuration; authentication; APIs; RBAC; RLS; validation; test data; backend deployment/accessibility.

Step 3 — Frontend readiness

All mock services identified; real API replacement points identified; environment variables configured; loading/empty/error states ready; responsive UI ready.

Step 4 — Connect

Replace mock API calls with real backend API calls.

Step 5 — Test module-by-module

Auth; RBAC; RLS/tenant isolation; Students; Student 360; Attendance; Fees; Exams; Results; Report cards; Communication; Parent dashboard; PTM; Forms; Import/export; Dashboards; loading/empty/error states.

Step 6 — Security verification

Unauthorized requests; role restrictions; school isolation; secret handling; server-side validation; RLS; data integrity.

Step 7 — Fix

Backend issues → Zainab. Frontend/UI/state issues → Mariya. Contract mismatch → both developers coordinate.

Step 8 — Deploy and smoke test

Verify environment variables, migrations, build and critical user flows.

> **FINAL FLOW**
>
> ALL COMPLETED MODULES → FULL SYSTEM INTEGRATION → END-TO-END TESTING → SECURITY VALIDATION → CRITICAL BUG FIXING → PRODUCTION VERIFICATION → DEPLOYMENT

15. Cross-Module Data Principle

“Enter once, use everywhere.”

- ✓ Student information should be reused across modules.
- ✓ Attendance connects to Student 360 and parent views.
- ✓ Fees connect to Student 360 and parent views.
- ✓ Results connect to Student 360, parent dashboard and report cards.
- ✓ Parent relationships determine parent access.
- ✓ School/tenant context applies across all modules.
- ✓ The frontend must not create a separate conflicting representation of core backend records.

STUDENT RECORD	ATTENDANCE	FEES	RESULTS
Student 360 Forms PTM Parent Dashboard	Student 360 Parent Dashboard Admin/Teacher views Export	Student 360 Parent Dashboard Admin Dashboard Export	Student 360 Parent Dashboard Report Cards Dashboards Export

The product specification describes the MVP as one connected system: a student added once should appear wherever that record is needed rather than being re-created independently by each module.

16. Final Responsibility Matrix

Area	Zainab	Mariya	Shared
Database	Primary	—	—
Supabase	Primary	—	—
APIs	Primary	Consumer	Contract
Business logic	Primary	—	—
RBAC/RLS	Primary	UI handling	Verification
Mock data	Contract owner	Primary user	Agreement

Area	Zainab	Mariya	Shared
UI	—	Primary	—
API contract	Backend implementation	Frontend consumer	Must agree
Testing	API/security	UI/E2E	Day 8
GitHub	Backend repo	Frontend repo	None required
Deployment	Backend/config	Frontend/config	Day 8
Integration	Backend side	Frontend side	Day 8

SHARED DOES NOT MEAN UNOWNED

The API contract, integration and Day 8 verification are coordinated responsibilities. Backend and frontend implementation ownership remains clear.

17. Final “One-Page Rules”

1. Zainab = Backend.
2. Mariya = Frontend.
3. Two separate GitHub repositories.
4. No password sharing.
5. API/Data Contract is the bridge.
6. Days 1–7 = independent development with mocks.
7. Mocks must match the agreed real API response shape.
8. Contract changes must be communicated before implementation continues.
9. Never expose service-role secrets.
10. Authentication + RBAC + RLS + server validation are mandatory.
11. Do not expand the MVP scope.
12. Full frontend ↔ backend integration happens on Day 8.
13. Day 8 includes integration, testing, security verification, fixing and deployment.
14. The existing MVP Feature Specification, API/Data Contract, Research document and 8-Day Plan remain the authoritative project sources.
15. This guide supplements those documents; it does not replace them.

THE WORKING AGREEMENT

BUILD MODULES IN PARALLEL → KEEP THE CONTRACT SYNCHRONIZED → COMPLETE MODULES → FULL INTEGRATION ON DAY 8 → TEST → SECURE → FIX → DEPLOY → PILOT

Source note

The project file index available in this workspace contained the Finalized MVP Feature Specification and Detailed Research & Product Blueprint. The API/Data Contract Specification and the 8-Day Development Plan were referenced by the supplied handoff brief but were not independently retrievable from the indexed project files during document generation. API examples, role values and the 8-day sequence in this guide therefore follow the exact material supplied in the handoff brief; they are not presented as independently re-derived from unavailable source files.